El Lenguaje

Memoria Dinámica en C++

Memoria Dinámica

- *El trabajo con memoria dinámica en C++ tiene muchas similitudes con lo que se hace en C.*
- *Pero también tiene sus diferencias.*
- *En C++ se utilizan las funciones new y delete para reservar y devolver memoria respectivamente.*
- *No necesitamos el casting de puntero que debe hacerse en C.*

Memoria Dinámica en C++

- *Para solicitar un elemento simple de memoria:*

```
pointer = new type;
Ej.:  base = new int;
```

- *También podemos reservar un bloque:*

```
pointer = new type[ elements ];
Ej.:  base = new int[ 100 ];
```

Memoria Dinámica

- *A partir de la reserva de memoria realizada con new el puntero apunta al área de memoria reservada, para devolverla usamos:*

```
delete a;
```

- *Es aconsejable usar delete SIEMPRE, especialmente si debemos repetir el ciclo de reservar memoria.*

```
#include <iostream>  // P03-01.cpp
#include <assert.h>

using namespace std;

int main() {
   int *a;
   char *b;
   float *c;
   struct stPunto { float x, y; } *d;

   a = new int;
   assert( a );
   b = new char;
   assert( b );
   c = new float;
   assert( c );
   d = new stPunto;
   assert( d );

   *a = 10;
   *b = 'a';
   *c = 10.32;
   d->x = 12; d->y = 15;

   cout << "a = " << *a << endl;
   cout << "b = " << *b << endl;
   cout << "c = " << *c << endl;
   cout << "d = (" << d->x << ", " << d->y << ")" << endl;
```

```
    delete a;
    delete b;
    delete c;
    delete d;

    return 0;
}
```

```
Salida:
a = 10
b = a
c = 10.32
d = (12, 15)
```

- *Para liberar memoria el puntero debe estar apuntando a la posición que se libera.*
- *assert() es una función ubicada en la biblioteca homónima que implica una afirmación, si no se cumple emite un mensaje y detiene el programa con abort().*

Arreglos en Memoria Dinámica

- *El problema habitual con los arreglos es que debemos conocer su tamaño en forma anticipada.*
- *No sucede lo mismo con los arreglos en memoria dinámica.*
- *Una forma común de reservar y liberar memoria para un arreglo es como se muestra a continuación.*

```
#include <iostream> // P03-02.cpp
using namespace std;
int main() {
  int **tabla;
  int n = 100;
  int m = 200;
  int i;
  // Arreglo de punteros a int
  tabla = new int*[n];
  // n arreglos de m ints
  for(i = 0; i < n; i++)
    tabla[i] = new int[m];
  tabla[21][33] = 123;      // Ejemplo de asignación
  cout << tabla[21][33] << endl;
  // Liberación de memoria, deshaciendo el camino hecho
  for(i = 0; i < n; i++) delete[] tabla[i];
  delete[] tabla;
  return 0;
}
```

```
Salida:
123
```

Memoria Dinámica

- *En C++ no tenemos la constante NULL.*
- *En su lugar simplemente utilizamos 0.*
- *La razón es que en C la definición de NULL es: (void *).*
- *C++ no permite la conversión implícita de void a otro tipo, en cambio el entero 0 si es convertible a cualquier tipo.*

Puntero constante vs Puntero a constante

- *Debemos diferenciar lo que es un puntero a constante, no podemos cambiar el contenido de la dirección que apunta.*
- *Otra cosa es puntero constante, no podemos cambiar el lugar adonde apunta.*
- *También podemos declarar punteros constantes a una constante, no poseen demasiada utilidad, pero es importante identificarlos al leer un programa.*

```
#include <iostream>  / P03-03.cpp
using namespace std;
int main( void ) {
  double pi = 3.1416926;
  double e = 2.71828;
  const double *num = &pi;  // Puntero a constante
  cout << "El valor de PI es: " << *num << endl;
  // Esto es un error, el valor no se puede modificar
  *num = 2.0;
   num = &e;    // Puede cambiar el lugar donde apunta
  cout << "El valor de e es: " << *num << endl;
  double *const statnum = &pi; // Puntero constante
  cout << "El valor de PI es: " << *statnum << endl;
  // Esto es un error, no puedo cambiar adonde apunta
  statnum = &e;

  // Puntero constante a constante
  const double *const pten = &pi; // Punt.
  pten = &pi;          // Esto da error
  *pten = 3.1;         // Esto también da error
}
```